"Messy Mashup" Music Genre Classification

1. Abstract / Executive Summary

This project addresses the **"Messy Mashup" Kaggle competition**, which challenges participants to perform robust music genre classification across 10 distinct genres under severe domain shift and noise. The training data comprises clean, isolated instrument stems, whereas the testing data consists of tempo-synchronized mashups constructed from multiple songs and obscured by **ESC-50 environmental noise**.

To tackle this, a dynamic data augmentation pipeline was engineered to simulate the test environment (utilizing tempo interpolation, multi-noise injection, and stem dropout). Three diverse models were trained: a custom **Bi-GRU CRNN**, an adapted **EfficientNet-B0**, and a **Fine-Tuned Audio Spectrogram Transformer (AST)**. The project highlights the critical importance of domain-specific data augmentation, with the adapted EfficientNet-B0 achieving a peak **Macro F1 score of 0.90252**.

2. Introduction

Problem Statement

The competition requires models to learn genre-invariant musical characteristics. The training set provides 1,000 clean songs split into four instrument stems (drums, vocals, bass, others). The hidden test set contains 3,020 noisy "mashups," created by cross-combining stems from different songs, synchronizing tempos, and adding synthetic environmental noise. The core challenge is bridging the domain gap between pristine stems and chaotic, noisy mixes.

Project Objective

The goal is to design a robust classification pipeline achieving a **Macro F1 score > 0.80**. Objectives include:

- Mastering **PyTorch** audio data pipelines.
- Experimenting with **Hugging Face AST** architectures.
- Building custom sequence models (**RNNs**).
- Leveraging **Weights & Biases (W&B)** for hyperparameter tracking.

Report Structure

- **Section 3:** Dataset & Augmentation Pipelines.
- **Section 4:** Core Architectures (Transformer, CNN, CRNN).

- **Section 5:** Performance & Comparative Analysis.
- **Section 6:** Conclusions & Future Work.

3. Dataset & Preprocessing

Dataset Description

- **Genres Stems:** 10 genres (blues, classical, country, disco, hiphop, jazz, metal, pop, reggae, rock), each containing 100 songs separated into drums.wav, vocals.wav, bass.wav, and other.wav.
- **ESC-50 Dataset:** 2,000 environmental noise clips spanning 50 classes.

Exploratory Data Analysis (EDA)

Training stems are uniform at **22050 Hz**. Test distribution analysis revealed a specific RMS energy footprint (**mean=0.14120**, **std=0.02697**) and conflicting tempos requiring synchronization. The dataset is perfectly balanced across classes.

Data Preprocessing

Audio was resampled dynamically to **22050 Hz** (or **16000 Hz** for AST). An **80/20 train/validation split** was enforced at the *song* level to prevent data leakage.

Data Augmentation

The methodology evolved in two phases to match the test distribution:

- **Phase 1 (Baseline):** 4.0s crops, random stem mixing, single ESC-50 noise clip (90% chance), and SpecAugment.
- **Phase 2 (Dynamic Generation):**
    - **Extended Context:** Increased crop length to 6s.
    - **Tempo Augmentation:** Random stretching/compressing (**0.85x to 1.15x**) using F.interpolate for speed.
    - **Stem Dropout:** 15% probability of omitting a stem.
    - **Multi-Noise:** 1 to 3 concurrent noise clips.

4. Modeling & Experimentation

4.1. Model 1: Fine-Tuned Transformer Model (Pretrained AST)

- **Architecture:** Applies a standard ViT architecture directly to audio spectrograms.
- **Salient Points:** Captures global context (long-range musical dependencies) better than local CNNs and is exceptionally robust to noise.
- **Fine-Tuning:** The first 50 parameters were frozen. The output was routed to a custom

head: `Dropout(0.5) -> Linear(768, 256) -> ReLU -> Dropout(0.3) -> Linear(256, 10)`.

4.2. Model 2: Custom Deep Learning Model (Adapted EfficientNet-B0)

- **Architecture:** Pre-trained vision architecture adapted for single-channel audio.
- **Salient Points:** Balanced depth and resolution scaling. Replacing the `Conv2d(3, ...)` with `Conv2d(1, ...)` allowed the model to excel at finding fine-grained localized spectral textures representing instrumental timbre.

4.3. Model 3: RNN-Based Model (Improved CRNN)

- **Architecture:** A custom Convolutional Recurrent Neural Network built from scratch.
- **Salient Points:** Traces the evolution of melody and rhythm via **Bi-GRU** sequence frames. **Squeeze-and-Excitation (SE)** blocks weight frequency channels, while **Attention Pooling** allows the model to "focus" on clean frames within noisy sequences.

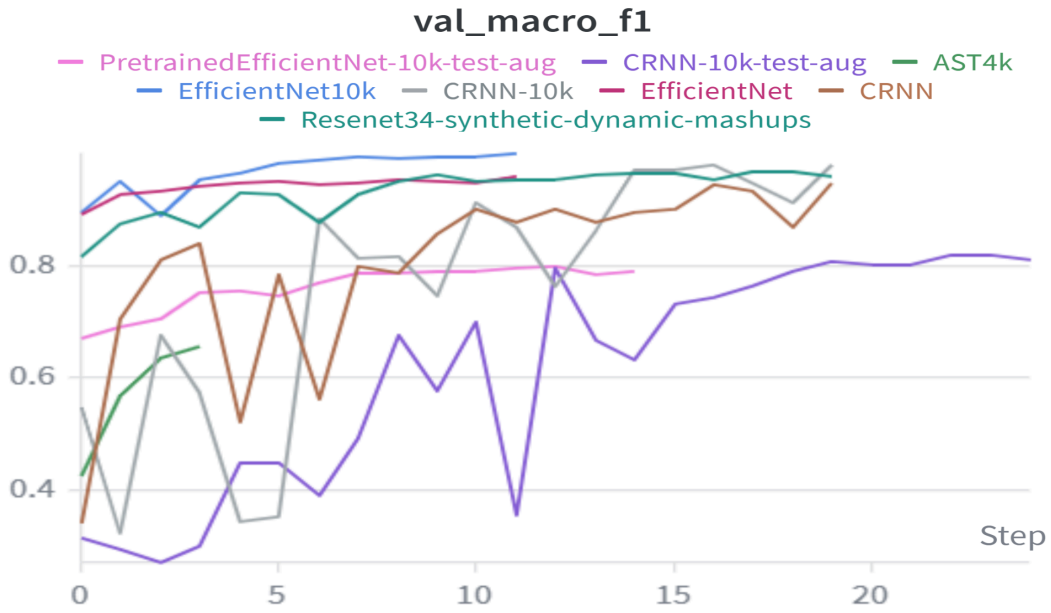
5. Performance & Comparative Analysis

Evaluation Metrics

The primary metric is the **Macro F1 Score**, providing a uniform average across all 10 classes to ensure robustness against minor imbalances.

Training Performance (W&B Summary)

Optimized using **Label Smoothing Cross-Entropy** and **MixUp** (`alpha=0.3`).



Model(wandb run name)	Data Augmentation	Initial LR	Epochs	Highest Val F1(public score)
BaselineEnsemble(CRNN-10k, EfficientNet, AST4k)	Phase 1 (Static 4s)	Various	vaires	0.76302
AST Pretrained (AST4k)	Phase 1	5e-5	3	0.6575
CRNN Custom(CRNN-10k-test-aug)	Phase 2 (Dynamic 6s)	1e-3	25	**0.86822**
EfficientNet-B0(Pretrained EfficientNet-10k-test-aug)	Phase 2 (Dynamic 6s)	5e-4	20	**0.90252**

Comparative Report

Data pipeline quality proved more vital than model complexity. Phase 2 synchronization and tempo shifts allowed single models to shatter the 0.80 barrier. **EfficientNet-B0** was the most stable performer, while the **CRNN** showed immense capacity for a scratch-built model.

6. Conclusion & Future Work

Key Learnings

Audio classification requires more than just "image" processing of spectrograms; tempo and rhythm are critical. Fine-tuning **Transformers (AST)** and adapting **Vision models** proved highly effective.

Challenges Faced

- **Computational Bottleneck:** Resampling 30s waveforms was slow; solved by cropping

before augmentation and using F.interpolate for a **more speedup**.
- **Domain Shift:** Extreme test noise caused overfitting, requiring heavy **MixUp** and **Dropout** regularization.

Future work could explore source separation as a pre-processing step during inference or utilizing more aggressive SpecAugment parameters to force reliance on broader structural cues.

7. References

1. Hugging Face Documentation: https://huggingface.co/learn/audio-course/en/chapter0/introduction.
2. PyTorch torchaudio Library.
3. Kaggle "Messy Mashup" Competition Guidelines.